AWS 3-Tier Production Web Architecture

End-to-End Scalable, Secure & High-Availability Infrastructure Implementation

Engineer: Aniket Nerkar

Domain: AWS Cloud & DevOps Engineering

Project: QuickLoan Web App Deploy

Project Executive Summary

This technical report documents the complete deployment of a production-grade, fault-tolerant, and auto-scaling 3-Tier Web Application ("Quick Loan System") hosted on Amazon Web Services (AWS). The architecture strictly isolates application tiers across dedicated public and private subnets within a custom Virtual Private Cloud (VPC), ensuring maximum security, compliance, and performance.

AWS VPC | Amazon EC2 | Amazon RDS MySQL | Application Load Balancer | Auto Scaling Group | Amazon S3 | CloudWatch & SNS | Nginx & PHP

1. Network Infrastructure & VPC Design

The network foundation is built upon a custom VPC (`my-quickloan-vpc`) configured across multiple Availability Zones in the `us-east-1` region. The architecture segregates public-facing resources (Bastion/Jump host, NAT Gateway, ALB) from internal workload components (Web/App EC2 instances and RDS database).

Component	Resource Name	Configuration / Target
VPC	`my-quickloan-vpc`	Custom CIDR block with DNS hostnames enabled
Internet Gateway	`quickloan-igw1`	Attached to VPC for public internet ingress/egress
NAT Gateway	`quickloan-natg1`	Deployed in Public Subnet for outbound private subnet traffic
Route Tables	`public-route`, `pvt-route`, `RDS-Pvt-rt`	Isolated routes directing public traffic to IGW & private to NAT

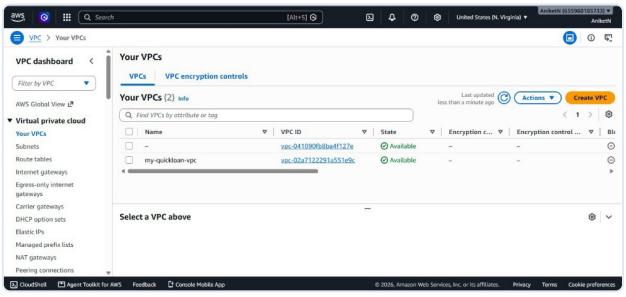


Figure 1.1: AWS VPC Console

Active `my-quickloan-vpc` deployed in `us-east-1`.

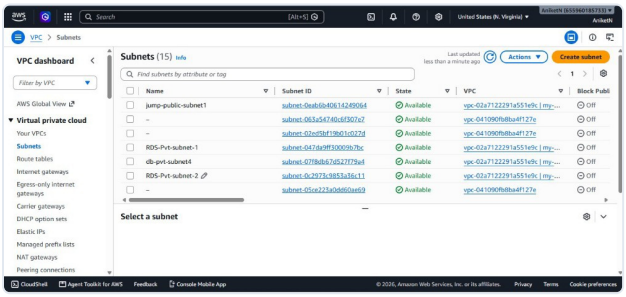


Figure 1.2: Subnets Infrastructure

Isolated Public, DB, and Application Subnets.

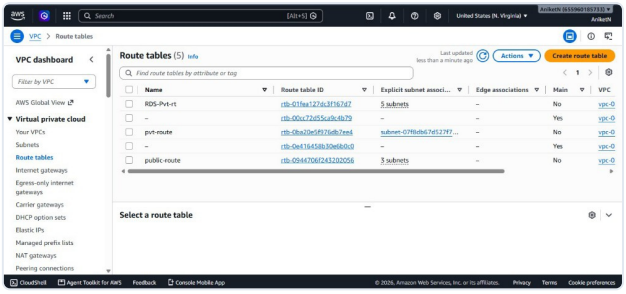


Figure 1.3: Route Tables Management

Public routes associated with IGW and private routes with NAT.

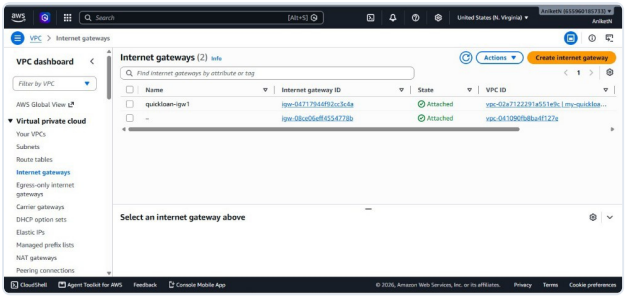


Figure 1.4: Internet Gateway Setup

Active `quickloan-igw1` attached to custom VPC.

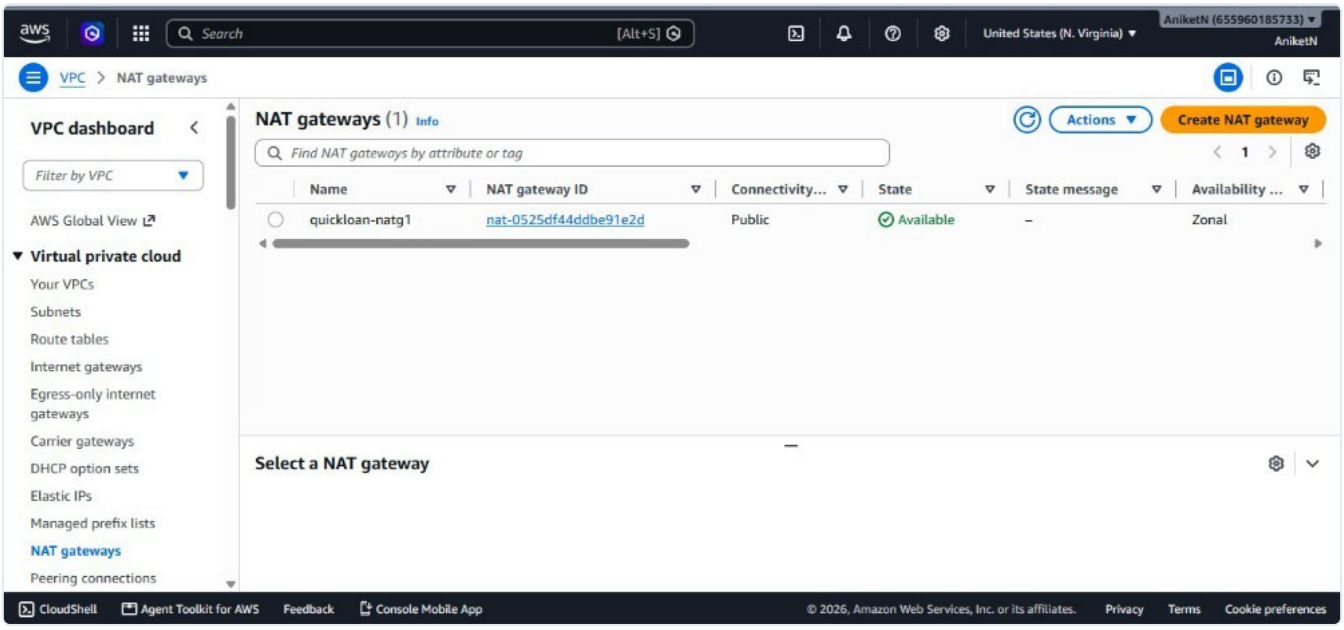


Figure 1.5: NAT Gateway Provisioning (`quickloan-natg1`)

Provides secure outbound internet connectivity for private instances to perform package updates and database synchronization without exposing them to direct internet incoming threats.

2. Compute Tier, Load Balancing & Auto Scaling

The compute infrastructure utilizes Amazon EC2 instances distributed across availability zones. An Elastic Application Load Balancer (ALB) routes incoming web traffic on HTTP Port 80 to healthy EC2 targets registered inside Target Group `TG1`.

- **Jump Server / Bastion Host:** Configured in `jump-public-subnet1` for secure SSH administrative access.
- **Launch Template (`web-template`):** Defines standardized EC2 configuration, AMI, security groups, and automated user-data initialization scripts.
- **Auto Scaling Group (`ASG1`):** Maintains high availability by automatically adjusting capacity based on real-time traffic and CPU utilization metrics.

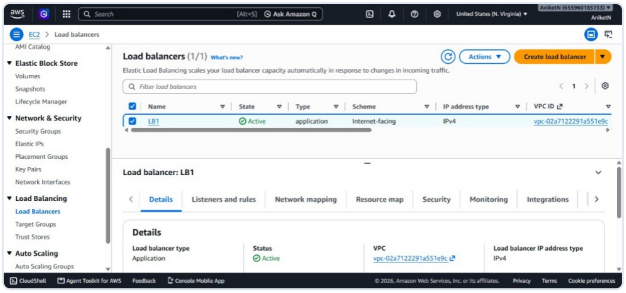


Figure 2.1: Active EC2 Instance Fleet

Running instances including Jump Server, Web ASG nodes, and DB server.

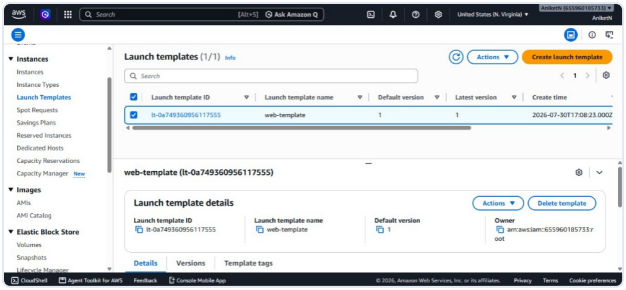


Figure 2.2: Launch Template Configuration

`web-template` used by ASG for dynamic instance spawning.

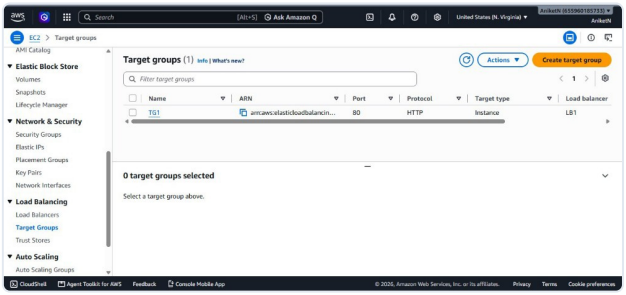


Figure 2.3: Application Load Balancer (LB1)

Internet-facing ALB distributing traffic across Availability Zones.

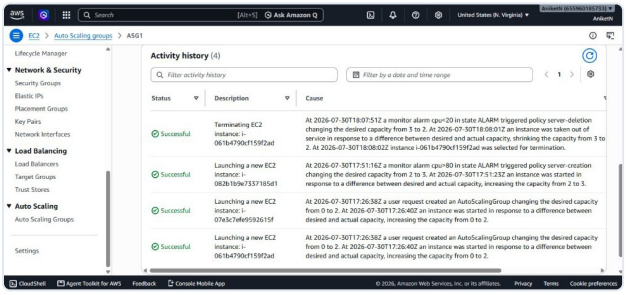


Figure 2.4: Target Group Setup (TG1)

Target Group monitoring EC2 web server health on Port 80.

3. CloudWatch Monitoring & Automated Scaling Policy

To handle variable load seamlessly, CloudWatch Alarms trigger automated scaling operations defined in the Auto Scaling Group `ASG1`:

- **Scale Out Policy (`cpu>80`):** Triggers when CPU utilization exceeds 80%, expanding capacity from 2 to 3 EC2 instances.
- **Scale In Policy (`cpu<20`):** Triggers when CPU drops below 20%, terminating redundant instances (e.g., terminating instance `i-061b4790cf159f2ad`) to optimize cloud spending.
- **Amazon SNS Notification (`web-topic`):** Sends real-time operational alerts to system engineers upon scaling events.

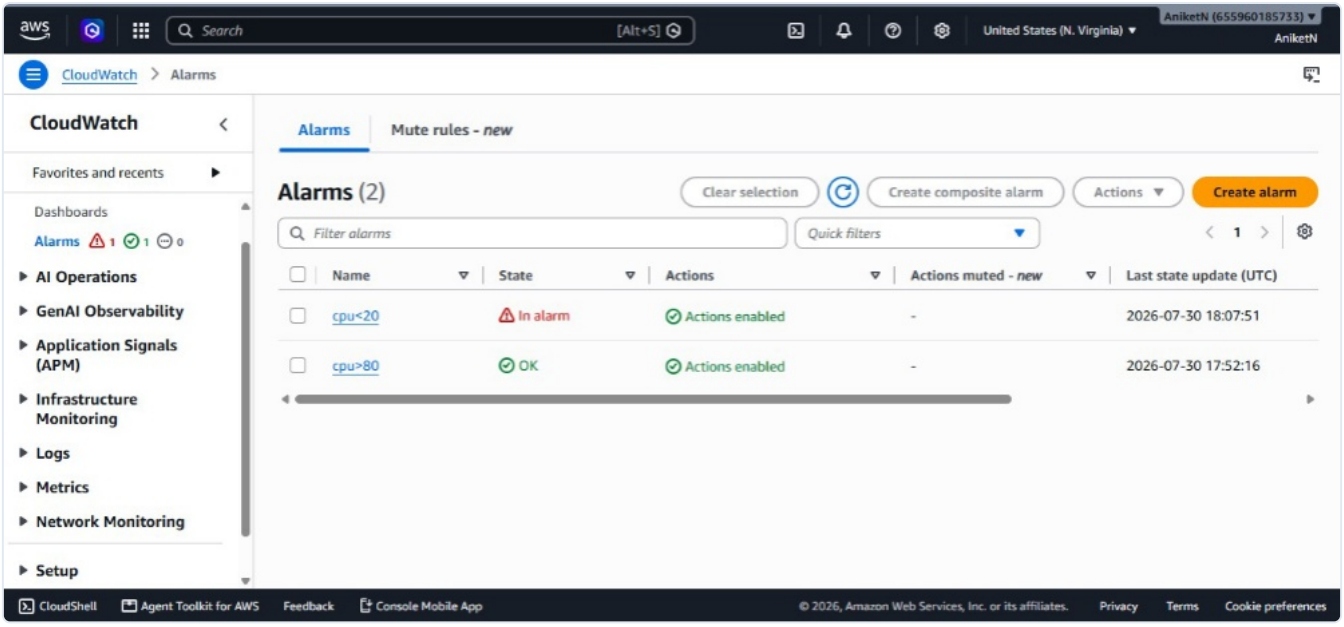


Figure 3.1: CloudWatch Alarms (`cpu<20` & `cpu>80`)

Active CloudWatch alarm triggers monitoring instance workload health.

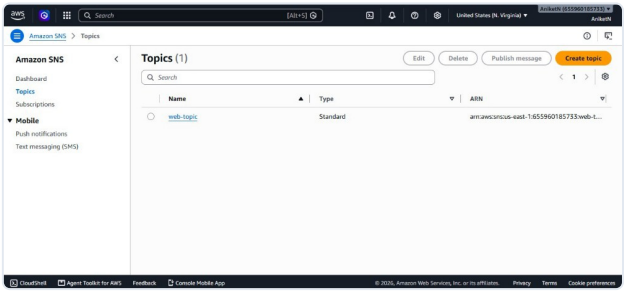


Figure 3.2: ASG Activity Logs

Successful launch and termination activity history of EC2 instances.

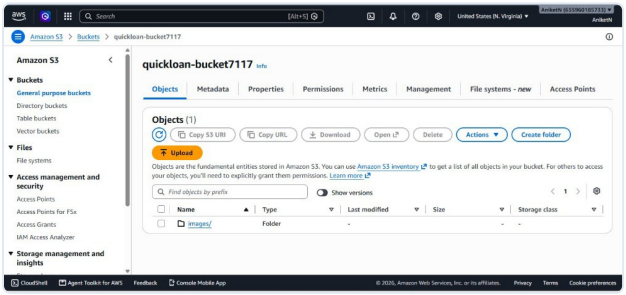


Figure 3.3: Amazon SNS Topic (`web-topic`)

Pub/Sub messaging service configured for operational notification alerts.

4. Database Tier & Data Persistence Layer

The database tier leverages Amazon RDS running a MySQL engine (`customers-db`) deployed across private subnets (`RDS-Pvt-subnet-1` and `RDS-Pvt-subnet-2`). Direct public access is disabled; connectivity is restricted strictly to security groups associated with web application servers.

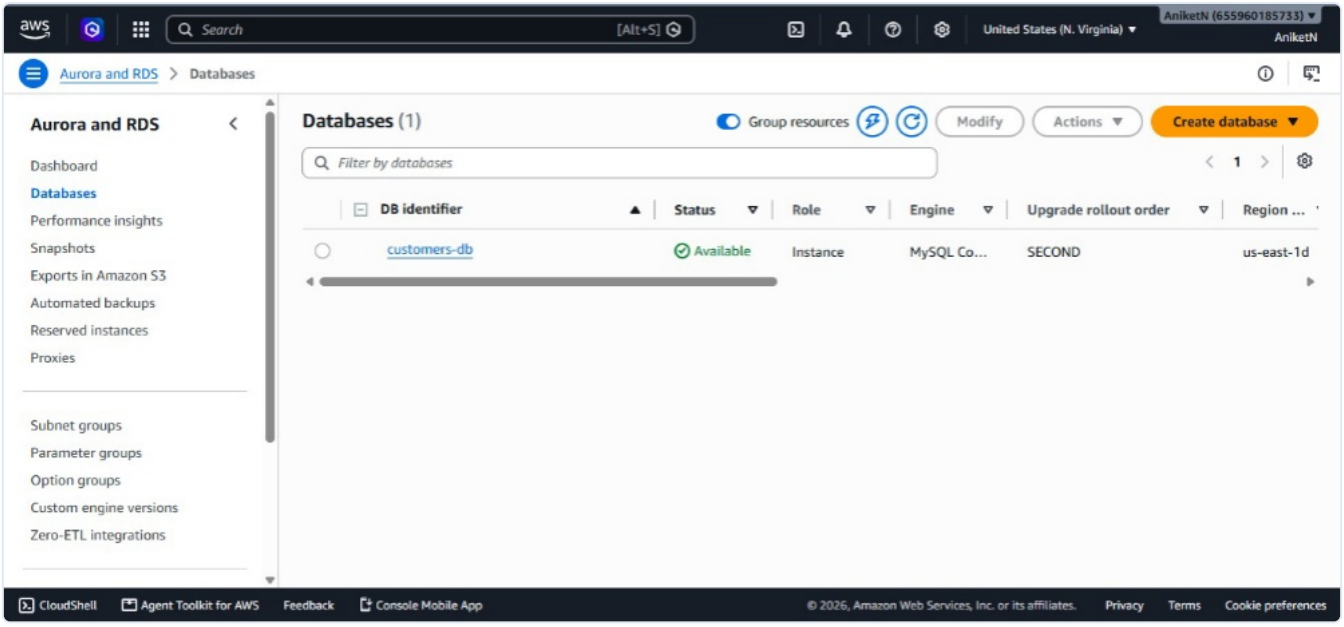


Figure 4.1: Amazon RDS MySQL Database Instance (`customers-db`)

Multi-AZ ready relational database instance operating in private isolated network.

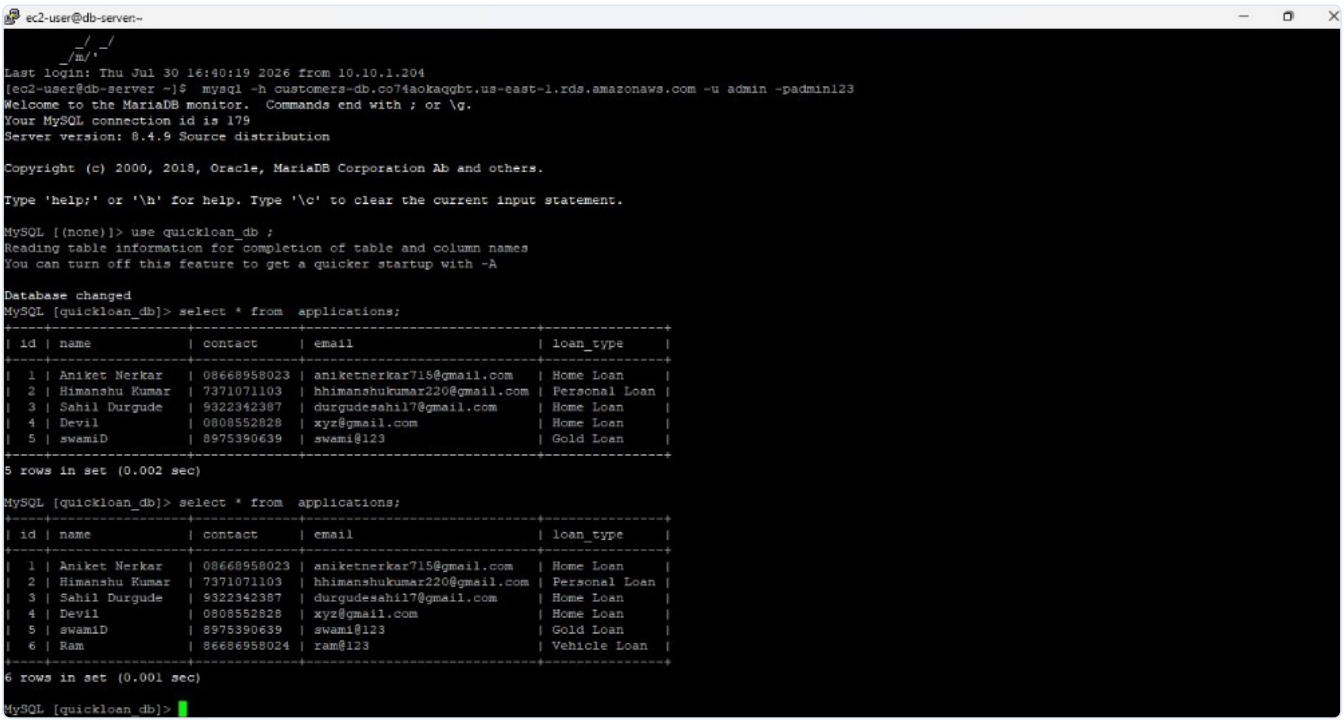


Figure 4.2: Direct MySQL CLI Validation & Live Records Query

Verifying persistent data insertion inside database `quickloan_db.applications` directly via remote MySQL CLI session.

5. Application Deployment & Live Verification

The web front-end is hosted on web servers initialized with Nginx and PHP. Static assets and media files are stored in Amazon S3 bucket `quickloan-bucket7117`. Code deployment was managed via SFTP/WinSCP protocols directly onto the server instance path `/home/ec2-user/nginx`.

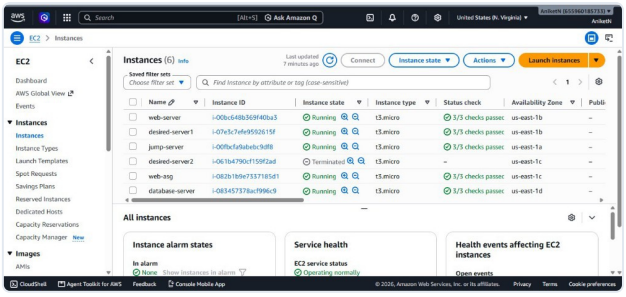


Figure 5.1: Amazon S3 Bucket Storage

Bucket `quickloan-bucket7117` holding web application assets.

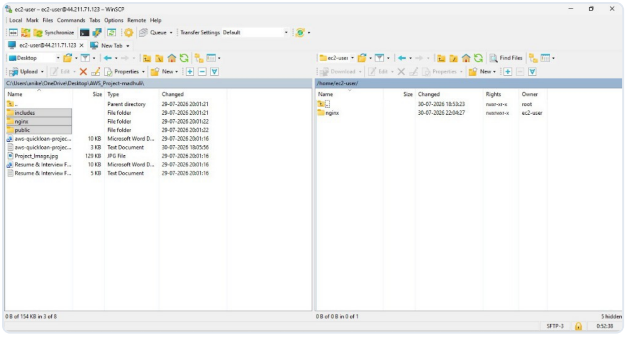


Figure 5.2: File Deployment via WinSCP

Deployment of PHP application scripts and Nginx configuration files.

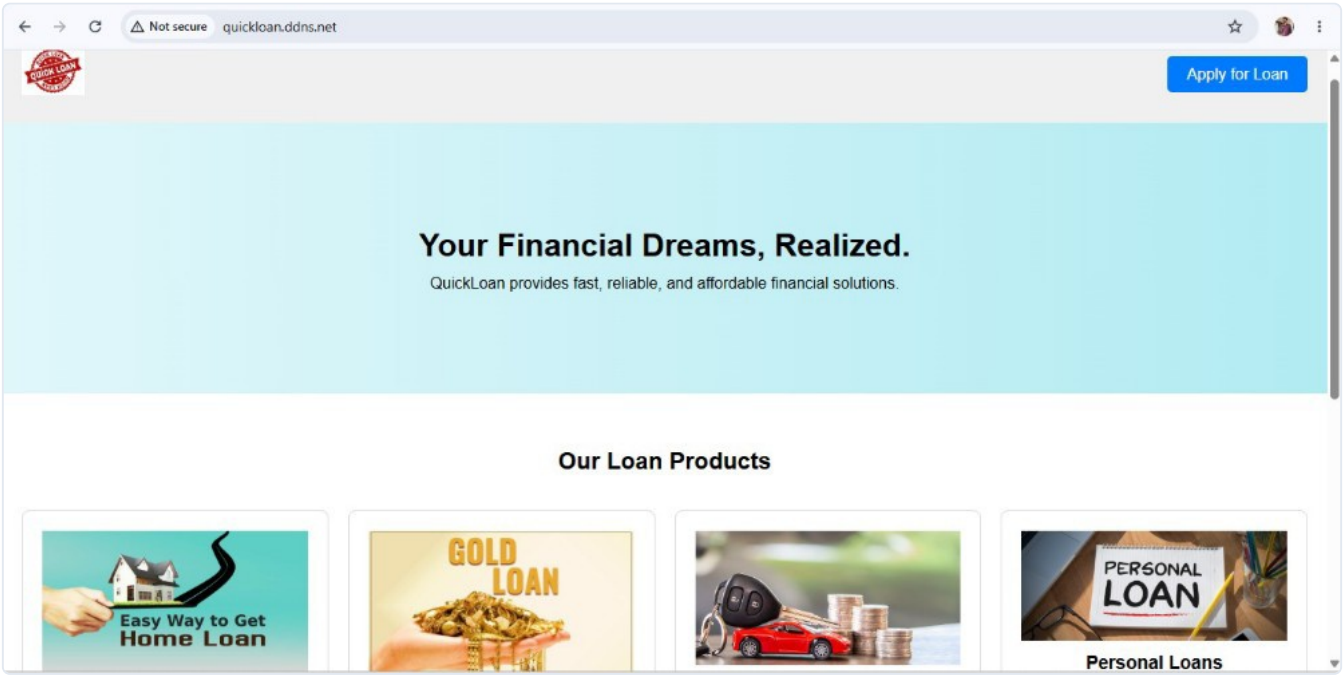


Figure 5.3: Live QuickLoan Web Portal (`quickloan.ddns.net`)

Publicly accessible web interface displaying loan products and navigation options.

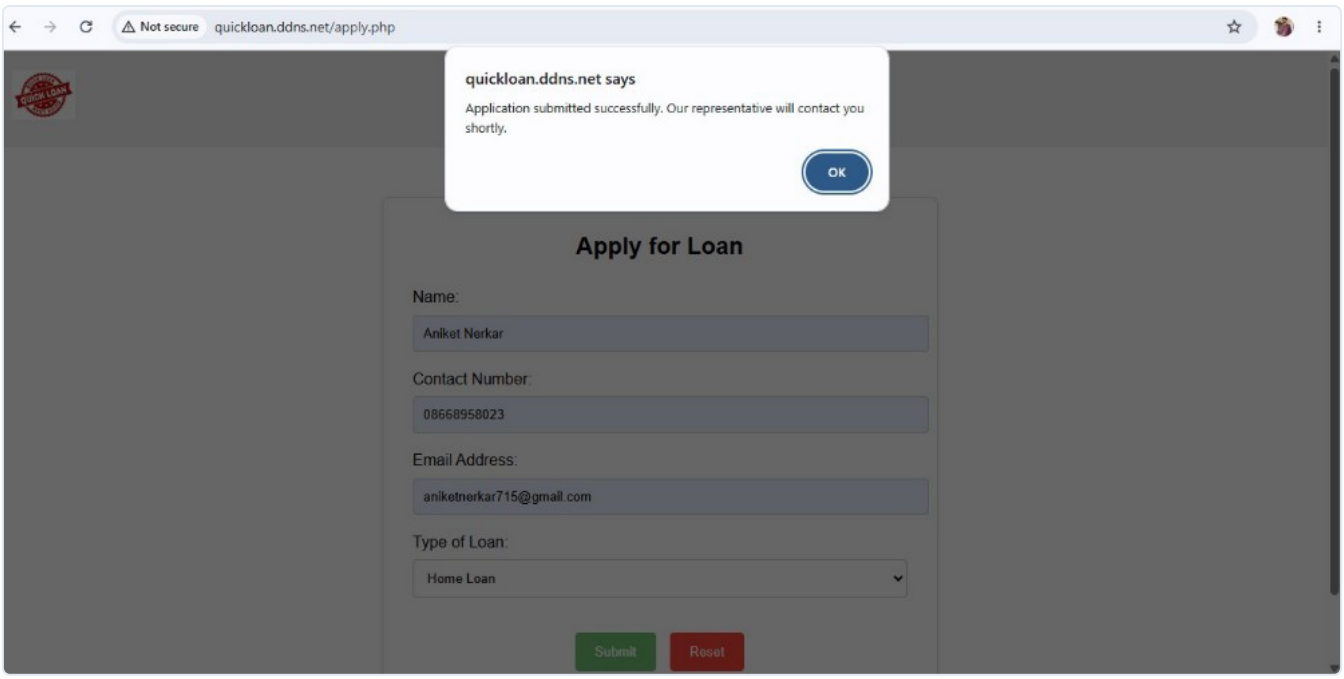


Figure 5.4: Successful Customer Loan Application Submission

User form submission instantly triggering backend validation and database record persistence into RDS.

Key Technical Accomplishments

- **High Availability & Fault Tolerance:** Architecture spans multiple Availability Zones behind an Application Load Balancer.
- **Automated Elasticity:** Dynamic Auto Scaling policies adjust EC2 capacity seamlessly in response to real-time traffic spikes.
- **End-to-End Security Isolation:** Database and private application workloads remain strictly inaccessible from the public internet.
- **Full-Stack Integration:** Dynamic data flowing seamlessly from frontend client forms into cloud-hosted MySQL databases.